

EXPERIMENT No. 01: Program to Implement Tic-Tac-Toe game

1.1 Objective

1.5 Procedure

1.2 Apparatus Required

1.6 Results

1.3 Pre-Requisite

1.7 Pre-Requisite Questions

1.4 Introduction

1.8 Post-Experimental Questions

1.1 Objective

Write a program to implement Tic-Tac-Toe game.

1.2 Apparatus Required

Python Language and Linux / Windows as OS, Jupyter Notebook

1.3 Pre-Requisite

Basic Python Programming

1.4 Introduction

Tic-tac-toe is a very popular game, so let's implement an automatic Tic-tac-toe game using Python. The game is automatically played by the program and hence, no user input is needed. Still, developing an automatic game will be lots of fun. Let's see how to do this. NumPy and random Python libraries are used to build this game. Instead of asking the user to put a mark on the board, the code randomly chooses a place on the board and put the mark. It will display the board after each turn unless a player wins. If the game gets drawn, then it returns -1.

Explanation: play_game() is the main function, which performs the following tasks :

- Calls create_board() to create a 3×3 board and initializes with 0.
- For each player (1 or 2), calls the random_place() function to randomly choose a location on board and mark that location with the player number, alternatively.

- Print the board after each move.
- Evaluate the board after each move to check whether a row or column or diagonal has the same player number. If so, displays the winner's name. If after 9 moves, there is no winner then displays -1.

1.5 Procedure

```
board = [" " for x in range(9)]
def print_board():
    row1 = "| {} | {} | {} |".format(board[0], board[1], board[2])
    row2 = "| {} | {} | {} |".format(board[3], board[4], board[5])
    row3 = "| {} | {} | {} |".format(board[6], board[7], board[8])
    print()
    print(row1)
    print(row2)
    print(row3)
    print()
def player_move(icon):
    if icon == "X":
        number = 1
    elif icon == "O":
        number = 2
    print("Your turn player {}".format(number))
    choice = int(input("Enter your move (1-9): ").strip())
    if board[choice - 1] == " ":
        board[choice - 1] = icon
    else:
        print()
        print("That space is taken!")
def is_victory(icon):
    if (board[0] == icon and board[1] == icon and board[2] == icon) or \
```

```
(board[3] == icon and board[4] == icon and board[5] == icon) or \
(board[6] == icon and board[7] == icon and board[8] == icon) or \
(board[0] == icon and board[3] == icon and board[6] == icon) or \
(board[1] == icon and board[4] == icon and board[7] == icon) or \
(board[2] == icon and board[5] == icon and board[8] == icon) or \
(board[0] == icon and board[4] == icon and board[8] == icon) or \
(board[2] == icon and board[4] == icon and board[6] == icon):
    return True
else:
    return False
def is_draw():
    if " " not in board:
        return True
    else:
        return False
while True:
    print_board()
    player_move("X")
    print_board()
    if is_victory("X"):
        print("X wins! Congratulations!")
        break
    elif is_draw():
        print("It's a draw!")
        break
    player_move("O")
    if is_victory("O"):
        print_board()
        print("O wins! Congratulations!")
        break
    elif is_draw():
```

```
print("It's a draw!")  
break
```

1.6 Results

```
| | | |
```

```
| | | |
```

```
| | | |
```

Your turn player 1

Enter your move (1-9): 1

```
| X | | |
```

```
| | | |
```

```
| | | |
```

Your turn player 2

Enter your move (1-9): 3

```
| X | | O |
```

```
| | | |
```

```
| | | |
```

Your turn player 1

Enter your move (1-9): 5

```
| X | | O |
```

```
| | X | |
```

```
| | | |
```

Your turn player 2

Enter your move (1-9): 6

```
| X | | O |
```

```
| | X | O |
```

```
| | | |
```

Your turn player 1

Enter your move (1-9): 9

```
| X | | O |
```

```
| |X|O|  
| | |X|
```

X wins! Congratulations!

1.7 Pre-Requisite Questions

1. What are the requirements for tic-tac-toe game?
2. How to create a tic-tac-toe game using Python?
3. Which algorithm is used for tic-tac-toe in Python?
4. What is the concept of tic-tac-toe?

1.8 Post-Experimental Questions

1. How to make a game in Python step by step?
2. What is the role of AI in tic-tac-toe?
3. What is the purpose of Tic Tac Toe?
4. What is the benefit of playing tic-tac-toe?

EXPERIMENT No. 02: Program to Implement A* Search Algorithm

2.1 Objective

2.5 Procedure

2.2 Apparatus Required

2.6 Results

2.3 Pre-Requisite

2.7 Pre-Requisite Questions

2.4 Introduction

2.8 Post-Experimental Questions

2.1 Objective

Write a program to implement A* Search Algorithm.

2.2 Apparatus Required

Python Language and Linux / Windows as OS, Jupyter Notebook

2.3 Pre-Requisite

Python Data Structures

2.4 Introduction

The A* search algorithm is a popular pathfinding and graph traversal algorithm used in various applications, including games and robotics. It finds the shortest path from a starting node to a target node using heuristics to guide its search.

- **Node Class:** Represents each position in the grid, storing its position, parent, and costs.
- **Heuristic Function:** Uses the Manhattan distance for calculating the heuristic cost.
- **A Function*:**
 - Initializes open and closed lists.
 - Processes nodes until it finds the goal or exhausts all options.
 - Explores neighbors and updates their costs.
 - Constructs the path if the goal is reached.

- **Example Usage:** Defines a grid and finds a path from the start to the goal.

Consider a square grid having many obstacles and we are given a starting cell and a target cell. We want to reach the target cell (if possible) from the starting cell as quickly as possible. Here A* Search Algorithm comes to the rescue. What A* Search Algorithm does is that at each step it picks the node according to a value-**'f'** which is a parameter equal to the sum of two other parameters – **'g'** and **'h'**. At each step it picks the node/cell having the lowest **'f'**, and process that node/cell. We define **'g'** and **'h'** as simply as possible below **g** = the movement cost to move from the starting point to a given square on the grid, following the path generated to get there. **h** = the estimated movement cost to move from that given square on the grid to the final destination. This is often referred to as the heuristic, which is nothing but a kind of smart guess. We really don't know the actual distance until we find the path, because all sorts of things can be in the way (walls, water, etc.). There can be many ways to calculate this **'h'** which are discussed in the later sections.

2.5 Procedure

```
def astarAlgo(start_node, stop_node):
    open_set = set(start_node)
    closed_set = set()
    g = {}
    parents = {}
    g[start_node] = 0
    parents[start_node] = start_node
    while len(open_set) > 0:
        n = None
        for v in open_set:
            if n == None or g[v] + heuristic(v) < g[n] + heuristic(n):
                n = v
```

```
if n == stop_node or graph_nodes[n] == None:
    pass
else:
    for (m, weight) in get_neighbours(n):
        if m not in open_set and m not in closed_set:
            open_set.add(m)
            parents[m] = n
            g[m] = g[n] + weight
        else:
            if g[m] > g[n] + weight:
                g[m] = g[n] + weight
                parents[m] = n
            if m in closed_set:
                closed_set.remove(m)
                open_set.add(m)
if n == None:
    print('Path Doesn\'t Exist!')
    return None
if n == stop_node:
    path = []
    while parents[n] != n:
        path.append(n)
        n = parents[n]
    path.append(start_node)
    path.reverse()
    print('Path Found : ', format(path))
    return path
    open_set.remove(n)
    closed_set.add(n)
print('Path Doesn\'t Exist!')
return None
```



```
def get_neighbours(v):
    if v in graph_nodes:
        return graph_nodes[v]
    else:
        return None
def heuristic(n):
    H_dist = {
        'A' : 10,
        'B' : 8,
        'C' : 5,
        'D' : 7,
        'E' : 3,
        'F' : 6,
        'G' : 5,
        'H' : 4,
        'I' : 1,
        'J' : 0
    }
    return H_dist[n]
graph_nodes = {
    'A' : [('B',6),('F',3)],
    'B' : [('C',3),('D',2)],
    'C' : [('D',1),('E',5)],
    'D' : [('C',1),('E',8)],
    'E' : [('I',5),('J',5)],
    'F' : [('G',1),('H',7)],
    'G' : [('I',3)],
    'H' : [('I',2)],
    'I' : [('E',5),('J',3)]
}
astarAlgo('A','J')
```

2.6. Results

Path Found : ['A', 'F', 'G', 'I', 'J']

['A', 'F', 'G', 'I', 'J']

2.7 Prerequisite Questions

1. What is the difference between a directed and an undirected graph?
2. What are nodes (vertices) and edges in a graph?
3. What is a path in a graph?
4. What makes an algorithm optimal? Is A* always optimal?

2.8 Post- Experimental Questions

1. How efficient was the search algorithm?
2. What was the algorithm's space complexity?
3. How did the algorithm perform on large data sets?
4. Were there any alternative algorithms that could have performed better?

EXPERIMENT No.03: Implement and demonstrate the FIND-S Algorithm

3.1 Objective

3.5 Procedure

3.2 Apparatus Required

3.6 Results

3.3 Pre-Requisite

3.7 Pre-Requisite Questions

3.4 Introduction

3.8 Post-Experimental Questions

3.1 Objectives:

Implement and demonstrate the FIND-S Algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file.

3.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

3.3 Pre-Requisite:

Basic Python Programming, Pandas Package

3.4 Introduction:

The Find-S algorithm is a machine learning algorithm used primarily in the domain of concept learning. It is designed for learning a conjunctive concept description by finding the most specific hypothesis that covers all positive examples, without covering any negative ones. Here's an overview of its working and background

The _S algorithm works on the assumption that the target concept can be represented as a conjunction of attributes. Its goal is to find the "**most specific hypothesis**" that matches all positive examples given in a training set. The hypothesis it generates should be as close to the actual concept as possible, without generalizing unnecessarily.

Working of the Find-S Algorithm:

The Find-S algorithm starts with the most specific hypothesis and then generalizes it incrementally to fit all positive training examples. It ignores negative examples.

1. Initialize the Hypothesis:

- Start with the most specific hypothesis, often denoted as $h = \langle \Phi, \Phi, \dots, \Phi \rangle$. Here, each attribute in h is initialized to a null value, indicating that no attribute value is accepted.

2. For each Positive Example:

- If the hypothesis h is consistent with the example, do nothing.
- If h is not consistent, update it by making h more general. Update only those attributes in h that differ from the attribute values in the positive example.

3. For each Negative Example:

- Skip, as Find-S does not utilize negative examples.

4. Output the Final Hypothesis:

- After examining all positive examples, output the most specific hypothesis h that matches all positive instances.

3.5 Procedure:

```
import csv

import pandas as pd
a = []
d=pd.read_csv("enjoysport.csv")
print(d)
with open('enjoysport.csv', 'r') as csvfile:
    for row in csv.reader(csvfile):
        a.append(row)
    print("\n The total number of training instances are : ",len(a))
    num_attribute = len(a[0])-1
    print("\n The initial hypothesis is : ")
```

```

hypothesis = ['0']*num_attribute
print(hypothesis)
for i in range(0, len(a)):
    if a[i][num_attribute] == 'yes':
        for j in range(0, num_attribute):
            if hypothesis[j] == '0' or hypothesis[j] == a[i][j]:
                hypothesis[j] = a[i][j]
            else:
                hypothesis[j] = '?'
    print("\n The hypothesis for the training instance {} is :\n"
        .format(i+1),hypothesis)
    print("\n The Maximally specific hypothesis for the training instance is ")
    print(hypothesis)

```

3.6 Results:

	sky	airtemp	humidity	wind	water	forecast	enjoysport
0	sunny	warm	normal	strong	warm	same	yes
1	sunny	warm	high	strong	warm	same	yes
2	rainy	cold	high	strong	warm	change	no
3	sunny	warm	high	strong	cool	change	yes

The total number of training instances are : 5

The initial hypothesis is :

['0', '0', '0', '0', '0', '0']

The hypothesis for the training instance 1 is :

['0', '0', '0', '0', '0', '0']

The hypothesis for the training instance 2 is :

['sunny', 'warm', 'normal', 'strong', 'warm', 'same']

The hypothesis for the training instance 3 is :

['sunny', 'warm', '?', 'strong', 'warm', 'same']

The hypothesis for the training instance 4 is :

['sunny', 'warm', '?', 'strong', 'warm', 'same']

The hypothesis for the training instance 5 is :

['sunny', 'warm', '?', 'strong', '?', '?']

The Maximally specific hypothesis for the training instance is

['sunny', 'warm', '?', 'strong', '?', '?']

3.7 Pre – Experimentation Questions

1. Explain specific hypothesis?
2. How does FIND S handle unseen or negative examples during hypothesis formation?
3. What is the structure of the hypothesis space?

3.8 Post-Requisite Questions

1. What is the primary goal of the Find-S algorithm?
2. Why does the Find-S algorithm only consider positive examples?
3. What are the key limitations of the Find-S algorithm?
4. In what scenarios would the Find-S algorithm fail to find a correct hypothesis?

EXPERIMENT No.04: Implement the Candidate-Elimination Algorithm

4.1 Objective	4.5 Procedure
4.2 Apparatus Required	4.6 Results
4.3 Pre-Requisite	4.7 Pre-Requisite Questions
4.4 Introduction	4.8 Post-Experimental Questions

4.1 Objectives:

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm to output a description of the set of all hypotheses consistent with the training examples.

4.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

4.3 Pre-Requisite:

Basic Python Programming, Pandas Package

4.4 Introduction:

The Candidate-Elimination algorithm is a concept-learning algorithm in machine learning, used to learn hypotheses from a given training set by incrementally refining two sets of hypotheses: the *most specific* and *most general* hypotheses. It starts with the most specific hypothesis (the empty set) and the most general hypothesis (the set of all possible hypotheses). As it processes each training example, it adjusts the boundaries of the specific and general hypotheses to exclude negative examples

and include positive ones, thereby narrowing the hypothesis space.

Step-by-Step Explanation:

1. Initialize the S and G sets:

- S (Specific boundary): The most specific hypothesis, initially the first positive example.
- G (General boundary): The most general hypothesis, initially the most general hypothesis (all possible hypotheses).

2. Iterate through each training example:

- If the example is positive, refine the set S by generalizing it to include the positive example and refine the set G by removing any hypotheses inconsistent with the positive example.
- If the example is negative, refine the set G by specializing it to exclude the negative example and refine S by removing inconsistent hypotheses.

3. Output the version space defined by S and G after processing all training examples.

4.5 Procedure:

```
import pandas as pd
df=pd.read_csv('EnjoySport.csv')
#df = df.drop(['sln0'], axis = 1)
concepts = df.values[:, :-1]
target = df.values[:, -1]
df.head()
def learn(concepts,target):
    specific_h = concepts[0].copy()
    general_h = [['?' for i in range(len(specific_h))] for i in range(len(specific_h))]
    for i,h in enumerate(concepts):
        if target[i] == "yes":
```



```
for x in range(len(specific_h)):
    if h[x] != specific_h[x]:
        specific_h[x] = '?'
        general_h[x][x] = '?'
if target[i] == "no":
    for x in range(len(specific_h)):
        if h[x] != specific_h[x]:
            general_h[x][x] = specific_h[x]
        else:
            general_h[x][x] = '?'
indices = [i for i,val in enumerate(general_h)
if val == ['?', '?', '?', '?', '?', '?']]
for i in indices:
    general_h.remove(['?', '?', '?', '?', '?', '?'])
return specific_h, general_h
s_final, g_final = learn(concepts, target)
print(f" Final S : {s_final}")
print(f" Final G : {g_final}")
```

4.6 Results:

Final S : ['sunny' 'warm' '?' 'strong' '?' '?']

Final G : [['sunny', '?', '?', '?', '?', '?'], ['?', 'warm', '?', '?', '?', '?']]

4.7 Pre – Experimentation Questions

1. What is the primary goal of the experiment?
2. What type of data will the algorithm be applied to?
3. What are the boundaries (S and G) expected to look like after processing?
4. How efficiently does the algorithm converge?

4.8 Post-Requisite Questions

1. What is the purpose of the Candidate Elimination Algorithm in machine learning?
2. Explain the concept of the Version Space in the context of Candidate Elimination.
3. What will happen if all training examples are positive? What does the version space look like in this case?
4. What types of hypothesis spaces are best suited for the Candidate Elimination Algorithm?

EXPERIMENT No.05: Demonstrate the working of the Decision Tree based ID3 Algorithm

5.1 Objective

5.5 Procedure

5.2 Apparatus Required

5.6 Results

5.3 Pre-Requisite

5.7 Pre-Requisite Questions

5.4 Introduction

5.8 Post-Experimental Questions

5.1

Objectives:

Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample. Built-in Java classes/API can be used to write the program.

5.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

5.3 Pre-Requisite:

Basic Python Programming, Pandas Package, numpy package, sklearn

5.4 Introduction:

The ID3 (Iterative Dichotomiser 3) algorithm is a fundamental algorithm used to generate a decision tree based on a given dataset.

Key Concepts

1. **Decision Tree:** A decision tree is a tree-like structure where each internal node represents a feature (or attribute), each branch represents a decision rule, and each leaf node represents the outcome (class label).
2. **Entropy:** Entropy measures the amount of uncertainty or impurity in a dataset. It is used to determine how informative a feature is for classifying data. The formula for entropy is:
$$\text{Entropy}(S) = -\sum_{i=1}^n p_i \log_2(p_i)$$
 Where p_i is the probability of class i in the dataset S .
3. **Information Gain:** Information gain is the reduction in entropy achieved by partitioning the dataset based on a particular feature. The ID3 algorithm selects the feature with the highest information gain for splitting the data at each step. The formula for information gain is:
$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$
 Where A is the feature, and S_v is the subset of S where feature A has value v .
4.
$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

Steps of the ID3 Algorithm

1. **Calculate Entropy:** Compute the entropy of the entire dataset.
2. **Information Gain Calculation:** For each feature, calculate the information gain using the formula above.
3. **Select the Best Feature:** Choose the feature with the highest information gain as the decision node for the current level.
4. **Split the Dataset:** Partition the dataset into subsets based on the selected feature's values.
5. **Recursive Process:** Repeat the process for each subset (recursively), treating each subset as a smaller dataset.
6. **Stopping Condition:** The recursion stops when:
 - a. All instances in a subset belong to the same class (pure subset).

- b. There are no more features to split, in which case a majority vote of the instances is used to determine the class.

5.5 Procedure:

```
import numpy as np
import pandas as pd
from sklearn import metrics
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# Reading the dataset
dataset = pd.read_csv('PlayTennis.csv')
features = ['Outlook', 'Temperature', 'Humidity', 'Wind']
X = dataset[features]
Y = dataset.PlayTennis

# Encoding categorical features
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
X_encoded = pd.DataFrame(encoder.fit_transform(X),
columns=encoder.get_feature_names_out(features))

# Splitting the dataset
X_train, X_test, y_train, y_test = train_test_split(X_encoded, Y, test_size=0.30,
random_state=100)

# Building the decision tree
```

```
dtree = DecisionTreeClassifier(criterion="entropy", random_state=100)
dtree.fit(X_train, y_train)
y_pred = dtree.predict(X_test)

# Function to classify a new instance
def classify_new_instance(outlook, temperature, humidity, wind, encoder):
    instance = [[outlook, temperature, humidity, wind]]
    instance_df = pd.DataFrame(instance, columns=features)
    instance_encoded = encoder.transform(instance_df)
    feature_names = encoder.get_feature_names_out(features)
    instance_encoded_df = pd.DataFrame(instance_encoded, columns=feature_names)
    prediction = dtree.predict(instance_encoded_df)
    return prediction[0]

# Predicting the class of a new instance
pred = classify_new_instance("Rain", "Mild", "High", "Strong", encoder=encoder)
print("Prediction:", pred)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

5.6 Results:

Prediction: No

Accuracy: 0.6

5.7 Pre – Experimentation Questions

1. What is the purpose of using the ID3 algorithm?
2. How does ID3 handle categorical and continuous data?
3. What dataset will you use for the experiment?
4. What is the role of entropy and information gain in ID3?

5.8 Post-Requisite Questions

1. What is the Decision Tree Algorithm?
2. Explain the difference between the CART and ID3 Algorithms.
3. List down the different types of nodes in Decision Trees.
4. List down the advantages of the Decision Trees.

EXPERIMENT No.06: Implementing the Backpropagation Algorithm

6.1 Objective

6.5 Procedure

6.2 Apparatus Required

6.6 Results

6.3 Pre-Requisite

6.7 Pre-Requisite Questions

6.4 Introduction

6.8 Post-Experimental Questions

6.1 Objectives:

Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

6.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

6.3 Pre-Requisite:

Basic Python Programming, Pandas Package, numpy package

6.4 Introduction:

The backpropagation algorithm is a supervised learning technique used in artificial neural networks to minimize the error by adjusting the weights of the network based on the output of each neuron. It is a key component of training neural networks, allowing them to learn from errors and improve their performance.

Key Concepts

1. **Neural Networks:** A neural network is composed of layers of interconnected neurons, each of which processes inputs and produces an output. These networks can be shallow (one hidden layer) or deep (multiple hidden layers).
2. **Feedforward Phase:** During this phase, input data is passed through the network, layer by layer, until it reaches the output layer. Each neuron applies a weighted sum of its inputs followed by an activation function to produce its output.
3. **Loss Function:** After the forward pass, the output of the network is compared to the actual target values using a loss function (e.g., Mean Squared Error for regression, Cross-Entropy for classification). The loss function quantifies how far off the network's predictions are from the actual values.
4. **Gradient Descent:** Backpropagation uses gradient descent to optimize the weights. The goal is to minimize the loss function by adjusting the weights in the direction that reduces the loss.
5. **Backpropagation Phase:** This is the core of the algorithm. It involves two main steps:
 - **Calculating Gradients:** The gradients of the loss function concerning each weight are computed using the chain rule of calculus. This involves propagating the error backward through the network from the output layer to the input layer.
 - **Updating Weights:** The weights are then updated using the computed gradients. Typically, a learning rate is used to control the size of the weight updates.

Steps of the Backpropagation Algorithm

1. **Initialization:** Randomly initialize the weights of the network.
2. **Feedforward:** For each training example:
 - Compute the output of the network by passing the inputs through the network (layer by layer).
3. **Calculate Loss:** Compute the loss using the predicted output and the actual target.
4. **Backpropagation:**
 - Compute the gradient of the loss with respect to the output of the network.
 - Propagate this error backward through the network to calculate the gradients

with respect to the weights.

5. **Update Weights:** Adjust the weights using the calculated gradients and the learning rate.

6. **Repeat:** Repeat the process for multiple epochs or until convergence is achieved.

6.5 Procedure:

```
import numpy as np
X = np.array([[2, 9], [1, 5], [3, 6]], dtype=float)
y = np.array([92, 86, 89], dtype=float)
X = X / np.amax(X, axis=0)
y = y / 100
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def derivatives_sigmoid(x):
    return x * (1 - x)
epoch = 1000
learning_rate = 0.6
inputlayer_neurons = 2
hiddenlayer_neurons = 3
output_neurons = 1
wh = np.random.uniform(size = (inputlayer_neurons,hiddenlayer_neurons))
bh = np.random.uniform(size = (1, hiddenlayer_neurons))
wo = np.random.uniform(size = (hiddenlayer_neurons,output_neurons))
bo = np.random.uniform(size = (1, output_neurons))
for i in range(epoch):
    net_h = np.dot(X, wh) + bh
    sigma_h = sigmoid(net_h)
    net_o = np.dot(sigma_h, wo) + bo
```



```
output = sigmoid(net_o)
deltaK = (y - output) * derivatives_sigmoid(output)
deltaH = deltaK.dot(wo.T) * derivatives_sigmoid(sigma_h)
wo = wo + sigma_h.T.dot(deltaK) * learning_rate
wh = wh + X.T.dot(deltaH) * learning_rate
print ("Input: \n" + str(X))
print ("Actual Output: \n" + str(y))
print ("Predicted Output: \n", output)
```

6.6 Results:

Input:

```
[[0.66666667 1.    ]
 [0.33333333 0.55555556]
 [1.    0.66666667]]
```

Actual Output:

```
[[0.92]
 [0.86]
 [0.89]]
```

Predicted Output:

```
[[0.89500204]
 [0.88083391]
 [0.8939834 ]]
```

6.7 Pre – Experimentation Questions

1. What is backpropagation?
2. What are the key components of a neural network?
3. What initialization method will you use for weights?
4. What are gradients, and why are they important?

6.8 Post-Requisite Questions

1. What is the chain rule in backpropagation?
2. What are some common loss functions used in backpropagation?
3. How does backpropagation work?

EXPERIMENT No.07: Calculate the Accuracy, Precision, and Recall for your data set.

7.1 Objective

7.5 Procedure

7.2 Apparatus Required

7.6 Results

7.3 Pre-Requisite

7.7 Pre-Requisite Questions

7.4 Introduction

7.8 Post-Experimental Questions

7.1 Objectives:

Assuming a set of documents that need to be classified, use the naïve Bayesian Classifier model to perform this task. Built-in Java classes/API can be used to write the program. Calculate the accuracy, precision, and recall for your data set.

7.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

7.3 Pre-Requisite:

Basic Python Programming, Pandas Package, performance matrix

7.4 Introduction:

The Naïve Bayesian Classifier is a probabilistic machine learning model based on

Bayes' theorem, used for classification tasks. It's particularly effective for text classification, such as spam detection or sentiment analysis, due to its simplicity and efficiency. The "naïve" aspect comes from the assumption that features (words in this case) are independent given the class label, which often holds well enough in practice despite being a simplification.

Evaluation Metrics

To evaluate the performance of the Naïve Bayesian Classifier, we use the following metrics:

1. **Accuracy:** The proportion of correctly classified documents over the total number of documents.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

2. **Precision:** The proportion of true positive results in relation to the total predicted positive results.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

3. **Recall:** The proportion of true positive results in relation to the actual positives.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negative}}$$

7.5 Procedure:

#Assuming a set of documents that need to be classified,

```
#use the naïve Bayesian Classifier model to perform this task.  
#Built-in Java classes/API can be used to write the program.  
# Calculate the accuracy, precision, and recall for your data set.
```

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.naive_bayes import MultinomialNB  
from sklearn import metrics  
  
msg = pd.read_csv('naivetext.csv', names=['message', 'label'])  
print("The dimension of the dataset", msg.shape)  
  
msg['labelnum'] = msg.label.map({'pos': 1, 'neg': 0})  
  
X = msg.message  
y = msg.labelnum  
  
print(X)  
print(y)  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)  
print("\n The total no of training data", y_train.shape)  
print("\n The total no of testing data", y_test.shape)  
  
vectorizer = CountVectorizer()  
X_train_dtm = vectorizer.fit_transform(X_train)  
X_test_dtm = vectorizer.transform(X_test)  
  
print("\n The words or token in text document\n")
```

```
print(vectorizer.get_feature_names_out())

clf = MultinomialNB()
clf.fit(X_train_dtm, y_train)

y_pred = clf.predict(X_test_dtm)

print("\n Accuracy of Classifier", metrics.accuracy_score(y_test, y_pred))
print("\n Confusion Matrix")
print(metrics.confusion_matrix(y_test, y_pred))
print("\n The value of precision", metrics.precision_score(y_test, y_pred))
print("\n The value of recall", metrics.recall_score(y_test, y_pred))
```

7.6 Results:

The dimension of the dataset (18, 2)

0	I love this sandwich
1	This is an amazing place
2	I feel very good about these beers
3	This is my best work
4	What an awesome view
5	I do not like this restaurant
6	I am tired of this stuff
7	I can't deal with this
8	He is my sworn enemy
9	My boss is horrible
10	This is an awesome place
11	I do not like the taste of this juice
12	I love to dance
13	I am sick and tired of this place

14 What a great holiday
15 That is a bad locality to stay
16 We will have good fun tomorrow
17 I went to my enemy's house today

Name: message, dtype: object

0 1
1 1
2 1
3 1
4 1
5 0
6 0
7 0
8 0
9 0
10 1
11 0
12 1
13 0
14 1
15 0
16 1
17 0

Name: labelnum, dtype: int64

The total no of training data (13,)

The total no of testing data (5,)

The words or token in text document

['about', 'am', 'amazing', 'an', 'awesome', 'bad', 'beers', 'boss', 'can', 'dance', 'deal', 'do', 'enemy', 'feel', 'good', 'great', 'holiday', 'horrible', 'house', 'is', 'like', 'locality', 'love', 'my', 'not', 'of', 'place', 'restaurant', 'sandwich', 'stay', 'stuff', 'that', 'these', 'this', 'tired', 'to', 'today', 'very', 'view', 'went', 'what', 'with']

Accuracy of Classifier 0.8

Confusion Matrix

[[3 0]

[1 1]]

The value of precision 1.0

The value of recall 0.5

7.7 Pre – Experimentation Questions

1. What is the objective of the classification?
2. How will you preprocess the documents?
3. How will you handle class imbalance?

7.8 Post-Requisite Questions

1. What data do you need for training and testing?
2. What metrics will you use to evaluate the classifier's performance?
3. How will you handle class imbalance in the dataset?
4. What challenges might arise when implementing this classifier?
5. What assumptions does the Naïve Bayes classifier make?

EXPERIMENT No.08: Demonstrate the working of SVM classifier

8.1 Objective

8.5 Procedure

8.2 Apparatus Required

8.6 Results

8.3 Pre-Requisite

8.7 Pre-Requisite Questions

8.4 Introduction

8.8 Post-Experimental Questions

8.1 Objectives:

Demonstrate the working of SVM classifier for a suitable dataset.

8.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

8.3 Pre-Requisite:

Basic Python Programming, Pandas Package, matplotlib, sklearn

8.4 Introduction:

Support Vector Machine (SVM) is a powerful supervised learning algorithm used primarily for classification tasks, although it can also be applied to regression. The

key concept behind SVM is to find the optimal boundary (hyperplane) that separates data points from different classes. Here's a breakdown of the main ideas related to SVM classifiers and suitable datasets:

Purpose of SVM:

- **Classification:** SVM is most commonly used for binary classification tasks, but it can be extended to multiclass classification.
- **Linear and Non-linear Classification:** SVM can classify data that is linearly separable (where a straight line or hyperplane can distinguish between classes) and data that is not linearly separable (using a technique called the kernel trick).

8.5 Procedure:

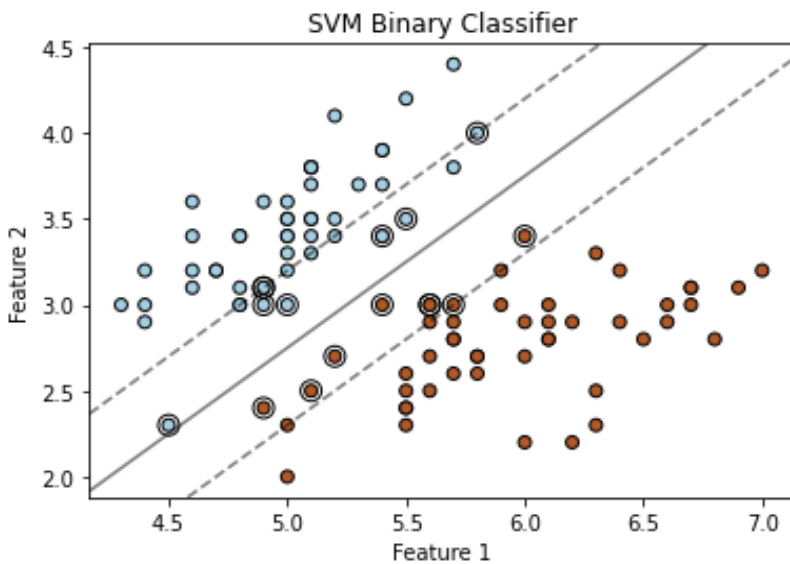
#AIM: Demonstrate the working of SVM classifier for a suitable dataset.

#Procedure:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
# Load the dataset (example: Iris dataset)
iris = datasets.load_iris()
X = iris.data[:, :2] # Consider only the first two features for simplicity
y = iris.target
# Select only two classes for binary classification
X = X[y != 2]
y = y[y != 2]
# Split the dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
# Create an SVM classifier
svm_classifier = SVC(kernel= 'linear')
# Train the classifier on the training data
svm_classifier.fit(X_train, y_train)
# Make predictions on the testing data
y_pred = svm_classifier.predict (X_test)
# Calculate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Plot the decision boundary and support vectors
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()
# Create a meshgrid to plot the decision boundary
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = svm_classifier.decision_function(xy).reshape(XX.shape)
# Plot the decision boundary and margins
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
ax.scatter(svm_classifier.support_vectors_[:, 0], svm_classifier.support_vectors_[:, 1],
s=100, linewidth=1, facecolors='none', edgecolors='k')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('SVM Binary Classifier')
plt.show()
```

8.6 Results:



8.7 Pre – Experimentation Questions

1. What is the problem you are trying to solve?
2. What is the nature of your data?
3. What type of SVM kernel will you use?
4. How will you evaluate performance?

8.8 Post-Requisite Questions

1. What is the main objective of a Support Vector Machine classifier?
2. How can SVM be extended to multi-class classification problems?
3. How would you handle imbalanced data when using SVM?
4. How can SVM be extended to multi-class classification problems?

EXPERIMENT No.09: Program to Implement k-Nearest Neighbor Algorithm

9.1 Objective

9.2 Apparatus Required

9.3 Pre-Requisite

9.4 Introduction

9.5 Procedure

9.6 Results

9.7 Pre-Requisite Questions

9.8 Post-Experimental Question

9.1 Objectives:

Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions. Java/Python ML library classes can be used for this problem.

9.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

9.3 Pre-Requisite:

Basic Python Programming, numpy Package, iris dataset, sklearn

9.4 Introduction:

The **K-Nearest Neighbors (KNN) algorithm** is a supervised machine learning method employed to tackle classification and regression problems. Evelyn Fix and Joseph Hodges developed this algorithm in 1951, which was subsequently expanded by Thomas Cover. The article explores the fundamentals, workings, and implementation of the KNN algorithm.

(K-NN) algorithm is a versatile and widely used machine learning algorithm that is primarily used for its simplicity and ease of implementation. It does not require any assumptions about the underlying data distribution. It can also handle both numerical and categorical data, making it a flexible choice for various types of datasets in classification and regression tasks. It is a non-parametric method that makes predictions based on the similarity of data points in a given dataset. K-NN is less sensitive to outliers compared to other algorithms.

The K-NN algorithm works by finding the K nearest neighbors to a given data point based on a distance metric, such as Euclidean distance. The class or value of the data point is then determined by the majority vote or average of the K neighbors. This approach allows the algorithm to adapt to different patterns and make predictions based on the local structure of the data.

9.5 Procedure:

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import numpy as np
```

```
dataset = load_iris()
X_train,X_test,y_train,y_test
=train_test_split(dataset["data"],dataset["target"],random_state = 0)
kn = KNeighborsClassifier(n_neighbors = 3)
kn.fit(X_train,y_train)
KNeighborsClassifier(n_neighbors=3)
prediction = kn.predict(X_test)
confusion_matrix(y_test, prediction)
```

9.6 Results:

```
array([[13, 0, 0],
       [ 0, 15, 1],
       [ 0, 0, 9]], dtype=int64)
```

9.7 Pre – Experimentation Questions

1. How is the dataset prepared?
2. How do you choose the value of 'k'?
3. What is the expected impact of different values of 'k' on model performance?

9.8 Post-Requisite Questions

1. How does locality affect k-NN performance?
2. How does k-NN compare to other machine learning algorithms?
3. How does k-NN handle large datasets in terms of time complexity?
4. What is the time complexity of making predictions with k-NN?

EXPERIMENT No.10:Implement the Locally Weighted Regression Algorithm

10.1 Objective

10.2 Apparatus Required

10.3 Pre-Requisite

10.4 Introduction

10.5 Procedure

10.6 Results

10.7 Pre-Requisite Questions

10.8 Post-Experimental
Questions

10.1 Objectives:

Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

10.2 Apparatus Required:

Python Language and Linux / Windows as OS, Jupyter Notebook

10.3 Pre-Requisite:

Basic Python Programming, numpy Package

10.4 Introduction:

Locally Weighted Regression (LWR), also known as *Locally Weighted Scatterplot Smoothing (LOWESS)* or *LOESS*, is a non-parametric, instance-based learning algorithm. Instead of fitting a single global model to the entire dataset, LWR focuses on estimating the target value for each query point by fitting a model locally to points within a certain neighborhood. This method is particularly useful when the relationship between the input and output variables varies significantly across the dataset, as it can adapt to these changes in a flexible, local manner.

Key Components of LWR:

1. **Instance-Based Approach:** LWR does not train a model in the traditional sense. Instead, it relies on training data stored in memory and makes predictions only when a new input is given, using data points near the query point.
2. **Local Weighting:** Each point in the neighborhood of the query point is assigned a weight based on its distance from the query point. Points closer to the query point are assigned higher weights, and this weight often comes from a Gaussian kernel or another weighting function. The kernel bandwidth parameter determines the size of the neighborhood.
3. **Weighted Linear Regression:** For each query point, a weighted linear regression model is fitted to the points in its neighborhood. The weights affect the linear regression model, giving more influence to points closer to the query point and less influence to those further away.

4. **Predictions:** Once the locally weighted model is fitted, the target value for the query point is estimated based on the fitted model.

10.5 Procedure:

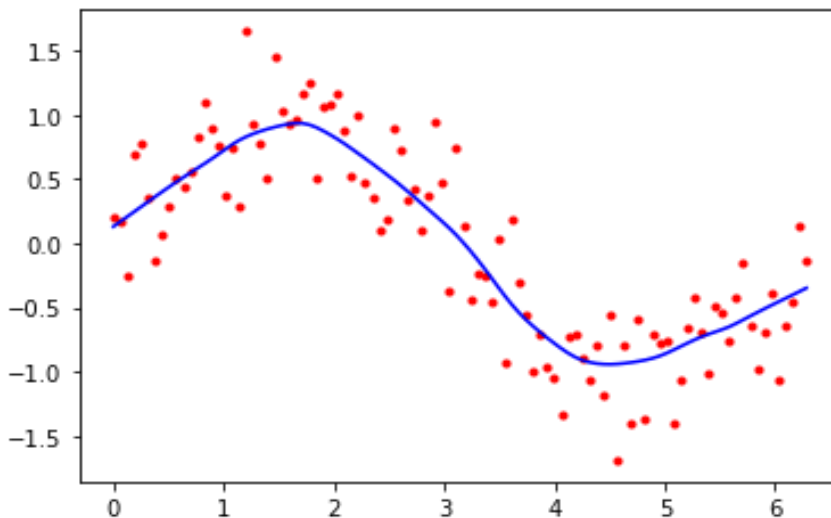
#AIM: Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

#Procedure:uy

```
from math import ceil
import numpy as np
from scipy import linalg
def lowess(x, y, f, iterations):
    n = len(x)
    r = int(ceil(f * n))
    h = [np.sort(np.abs(x - x[i]))[r] for i in range(n)]
    w = np.clip(np.abs((x[:, None] - x[None, :]) / h), 0.0, 1.0)
    w = (1 - w ** 3) ** 3
    yest = np.zeros(n)
    delta = np.ones(n)
    for iteration in range(iterations):
        for i in range(n):
            weights = delta * w[:, i]
            b = np.array([np.sum(weights * y), np.sum(weights * y * x)])
            A = np.array([[np.sum(weights), np.sum(weights * x)],
                          [np.sum(weights * x), np.sum(weights * x * x)]])
            beta = linalg.solve(A, b)
```

```
yest[i] = beta[0] + beta[1] * x[i]
residuals = y - yest
s = np.median(np.abs(residuals))
delta = np.clip(residuals / (6.0 * s), -1, 1)
delta = (1 - delta ** 2) ** 2
return yest
def main():
    import math
    n = 100
    x = np.linspace(0, 2 * math.pi, n)
    y = np.sin(x) + 0.3 * np.random.randn(n)
    f = 0.25
    iterations = 3
    yest = lowess(x, y, f, iterations)
    import matplotlib.pyplot as plt
    plt.plot(x,y,"r.")
    plt.plot(x,yest,"b-")
main()
```

10.6 Results:



10.7 Pre – Experimentation Questions

1. What is the purpose of using Locally Weighted Regression?
2. Define regression?
3. Explain instance base learning?

10.8 Post-Requisite Questions

1. What is Locally Weighted Regression (LWR)?
2. How does the concept of 'locality' work in LWR?
3. What are some typical applications of Locally Weighted Regression?